

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

Есть ли «новенькие»?

MS Teams - on-line общение

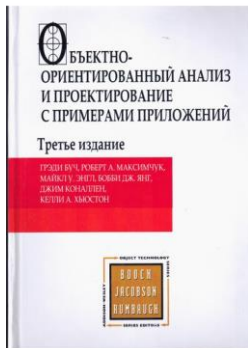
- код команды **8url2s2**
- присоединитесь, изучайте материалы сем.1
- установите человеческие аватарки
 - ваше лицо
 - хорошо различимо

Материал 1-го семестра нужен

- объект
- тип
- экземпляр (типа)
- инвариант (типа)
- контракт
- ...
- адрес
- указатель
- ссылка
- оператор
- класс
- структура
- ...

ваши вопросы

Что почитать



«Объектно-ориентированный анализ и проектирование с примерами приложений»
Гради Буч, и все-все-все...



«Язык программирования C++. Базовый курс»
Стенли Б. Липпман, Жози Лажойе, Барбара Э. Му

Предпосылки объектного подхода

- сложность систем ↑ (реальность)
 - по функционалу
 - по устройству
- стоимость разработки ПО ↓ (хотелка)
 - стоимость «железа» ↓
 - повторное использование кода
 - дешевые программисты

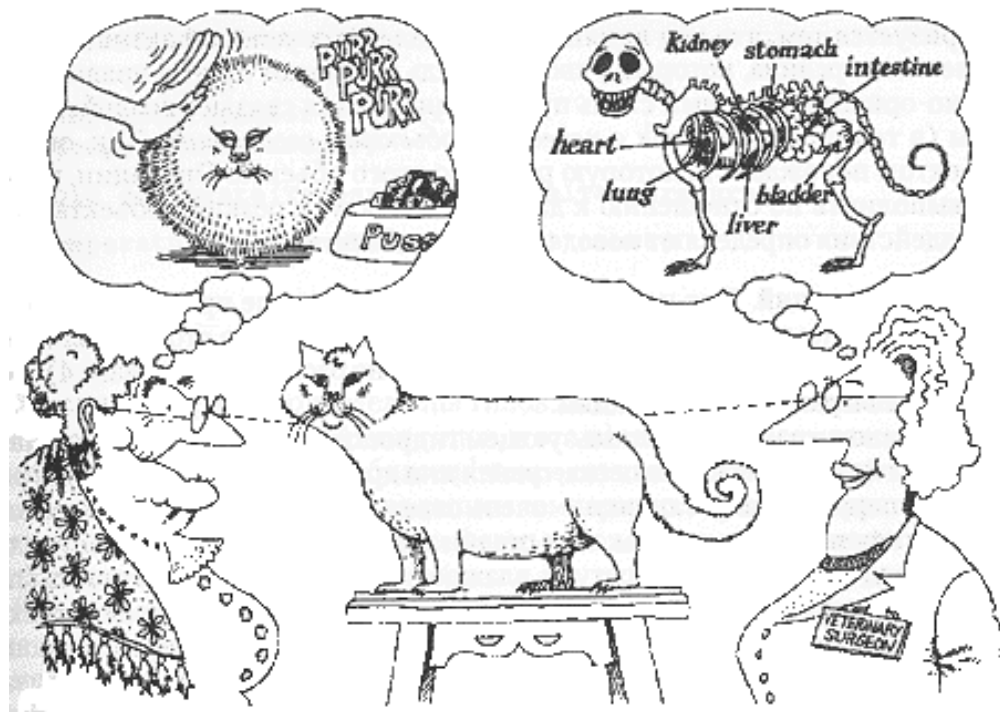
Идея объектного подхода

- программа моделирует реальность
- классы моделируют концепции реального и программного мира, экземпляры - объекты
- существуют отношения концепций
- язык C++ поддерживает моделирование концепций и некоторых типов отношений с помощью пользовательских типов

Абстракция

Выделяет важное

- свойства/данные
- поведение



Реализация повторного использования кода

создание новых типов на основе существующих

- композиция (агрегация)
- наследование

Композиция (отношение)

- способ создания новых типов
- НОВЫЙ ТИП включает поля существующих ТИПОВ
 - встроенные
 - пользовательские (структуры и классы)

Композиция (пример)

```
class Man {  
    ...  
    private:  
        string name_;  
        string secName_;  
};
```

```
class Student {  
    ...  
    private:  
        Man    man_;  
        Group  group_;  
};
```

Наследование (отношение)

- способ создания новых типов
- позволяет явно отражать общность типов (новый тип наследует члены)

Понятия иерархии наследования

- базовый (родительский) класс
- производный (дочерний) класс
- предок, потомок
- основание для иерархии
 - данные
 - интерфейс (поведение, ответственность)

Наследование (пример)

```
class Man {  
    ...  
    protected:  
        string name_;  
        string secName_;  
};
```

```
class Student  
    : public Man {  
    ...  
    private:  
        Group group_;  
};
```

Синтаксис наследования

```
class Derived
    : [private|protected|public] Base {
    // class body
};
```


Умолчательный спецификатор типа наследования

- `private` для классов
- `public` для структур
- спецификатор рекомендуется указывать явно

пример:

```
class Derived  
    : public Base {  
    ...  
};
```

Повышающее преобразование типа

- повышающее приведение типа – приведение указателя (ссылки) производного типа к указателю (ссылке) базового типа
- при обращении через указатель или ссылку с экземпляром производного типа можно обращаться как с экземпляром базового типа

Повышающее преобразование (пример)

```
// class Student : public Man  
// class Student student;
```

```
Man* pMan (&student) ;
```

```
...
```

```
pMan->name () ;
```

```
...
```

Student является Man, поэтому Student* можно
ИСПОЛЬЗОВАТЬ как Man*

Повышающее преобразование (пример)

```
// class Student : public Man  
  
void printInfo(ostream& ostr, Man& man) ;  
...  
Student me;  
...  
printInfo(cout, me) ;  
...
```

Подстановочный критерий (нестрогая формулировка)

- в любой точке вызова базового типа м.б. использован производный тип

или

- любой экземпляр производного типа является допустимым экземпляром родительского типа

ваши вопросы

Конструирование производных объектов

- конструктор базового класса д.б. вызван
- умолчательные конструкторы базовых типов м.б. вызваны неявно компилятором

Список инициализации (пример)

```
Student::Student(const string& name, const  
    string& secname, const Group& group)  
    : Man(name, secname)  
      , group_(group) {  
    // ...  
}
```


Конструктор производного класса

- должен
 - инициализировать базовый тип
 - инициализировать собственные поля
- непосредственная инициализация полей базового типа невозможна

Порядок вызова конструкторов

- по всей цепочке иерархии наследования, начиная от корневого базового типа и до текущего
- на каждом уровне вызывается
 - конструктор базового типа
 - конструкторы полей

Деструктор

- каждый класс содержит единственный деструктор (без аргументов)
- компилятор гарантирует вызов всех деструкторов (по всей иерархии наследования)

Порядок вызова деструкторов

- обратный порядку вызова конструкторов
- по всей цепочке иерархии наследования, начиная от текущего и до корневого базового типа
- на каждом уровне вызывается
 - деструкторы полей
 - деструктор базового типа

Замещение метода

- в производном типе м.б. определен метод с именем метода базового класса
- переопределение перегруженного имени функции базового типа скрывает все остальные версии в производном типе

Изменение интерфейса (в т.ч. замещение)

изменение интерфейса базового типа посредством модификации сигнатуры и/или типа возвращаемого значения означает, что тип используется не тем способом, для которого обычно предназначается наследование

Доступ к скрытым именам

осуществляется через полное имя

пример:

```
void Derived::print(ostream& ostr)
{
    Base::print(ostr) ;
    ...
}
```

Функции, которые не наследуются

- конструкторы
- деструктор
- **`operator=`**

Конструктор копирования при наследовании (пример)

```
Der::Der(const Der& copy)
    : Base(copy)
    // инициализация полей экземпляра
{
    // дополнительная работа
    // конструктора
}
```

Оператор присваивания при наследовании (пример)

```
Der& Der::operator=(const Der& obj) {  
    Base::operator=(obj) ;  
    // оставшаяся работа оператора  
}
```

Статические функции (при наследовании)

- наследуются производными типа
- переопределение скрывает все остальные перегруженные версии в базовом типе
- т.е. аналогично обычным функциям

Выбор между композицией и наследованием

- композиция
 - отношение «has-a» (b содержит a)
 - повторное использование кода
- наследование
 - отношение «is-a» (b является a)
 - повторное использование интерфейса

Закрытое наследование

- лучше использовать композицию
- реализует «ограниченную реализацию»
- пользователь не имеет доступа к базовой функциональности
- экземпляр производного типа не может интерпретироваться как экземпляр базового

Закрытое наследование (пример)

```
// in base.h
```

```
class Base {  
    public:  
        int aF();  
        int aF(int a);  
        void bF();  
  
    ...  
};
```

```
// in der.h
```

```
class Der  
    : private Base {  
    public:  
        using Base::aF;  
        using CBase::bF;  
  
    ...  
};
```

Защищенность

- ключевое слово `protected`
- защищенный член ведет себя как
 - открытый для членов производных типов
 - закрытый для всех остальных функций

Защищенное наследование

- реализует
 - открытую реализацию для производных и дружественных типов
 - «ограниченную реализацию» для других типов
- на практике используется редко

Матрица доступа

	методы и друзья класса	методы и друзья производных	пользователи
public	есть	есть	есть
protected	есть	есть	нет
private	есть	нет	нет

Рекомендации

- делайте все данные `private`, обеспечивая доступ к ним через методы
- используйте `protected` для методов реализации, которые могут использоваться в производных классах

Виды наследования

- одиночное наследование
один непосредственный базовый класс
- множественное наследование
более одного непосредственного базового
класса

Множественное наследование (синтаксис)

в списке базовых классов через запятую
указываются все базовые классы

пример:

```
class Derived
    : public Base1
    , public Base2 {
    // ...
```

Проблемы множественного наследования

- неоднозначности имен (членов и методов)
- повторяющиеся базовые классы (ромбовидная схема наследования)
- размещение данных в памяти
- порядок и количество вызовов методов

Виртуальное наследование

- должно решать проблему ромба
- добавляет новые «грабли»

пример:

```
class Derived
    : virtual public Base1
    , virtual Base2 {
    // ...
```

Использование множественного наследования

- избегайте без наличия весомай причины и полного понимания работы механизмов языка
- используйте для наследования нескольких интерфейсов

ваши вопросы

Связывание

- соотнесение вызова функции с телом функции
- раннее связывание
 - реализуется компилятором и компоновщиком до запуска программы (при «сборке»)
- позднее (динамическое) связывание
 - осуществляется в процессе исполнения программы в зависимости от фактического типа экземпляра

Виртуальная функция

- объявляется виртуальной в базовом классе с помощью ключевого слова `virtual`
- связывается динамически при косвенном использовании объекта базового класса
- остается виртуальной во всех производных классах (достаточно одного объявления в базовом классе)

Переопределение виртуальной функции

- повторное определение виртуальной функции в производном классе
- используйте `override` в производных классах для контроля перекрытия
- используйте модификатор `final` для запрета дальнейшего переопределения

Использование `virtual`

- избегайте использования `virtual` в определениях функций
- избегайте использования `virtual` в объявлениях производных классов

Виртуальная функция (пример)

```
class Man {  
    public:  
        virtual void writeTxt(ostream& os) const;  
        // ...  
};
```

Виртуальная функция (пример)

```
class Student
: public Man {
public:
    void writeTxt(ostream& os) const override;
};
```

```
class Tutor
: public Man {
public:
    void writeTxt(ostream& os) const final;
};
```

Расширяемость

- универсальный код
 - взаимодействует только с интерфейсом типа (т.е. базового класса)
- возможно добавление новых производных классов, без изменения универсального кода

Поиск тела виртуальной функции

- компилятор гарантирует наличие тела для виртуальной функции
- если виртуальная функция не переопределяется в классе, то вызывается «ближайшее» в иерархии наследования определение

Расширяемость (пример)

```
// Student : public Man, Tutor : public Man

void printStat(ostream& os,
    const vector<Man*> peoples)
{
    for (const auto& p : peoples) {
        p->writeTxt(os);
    }

    ...
}
```

Типичная реализация позднего связывания

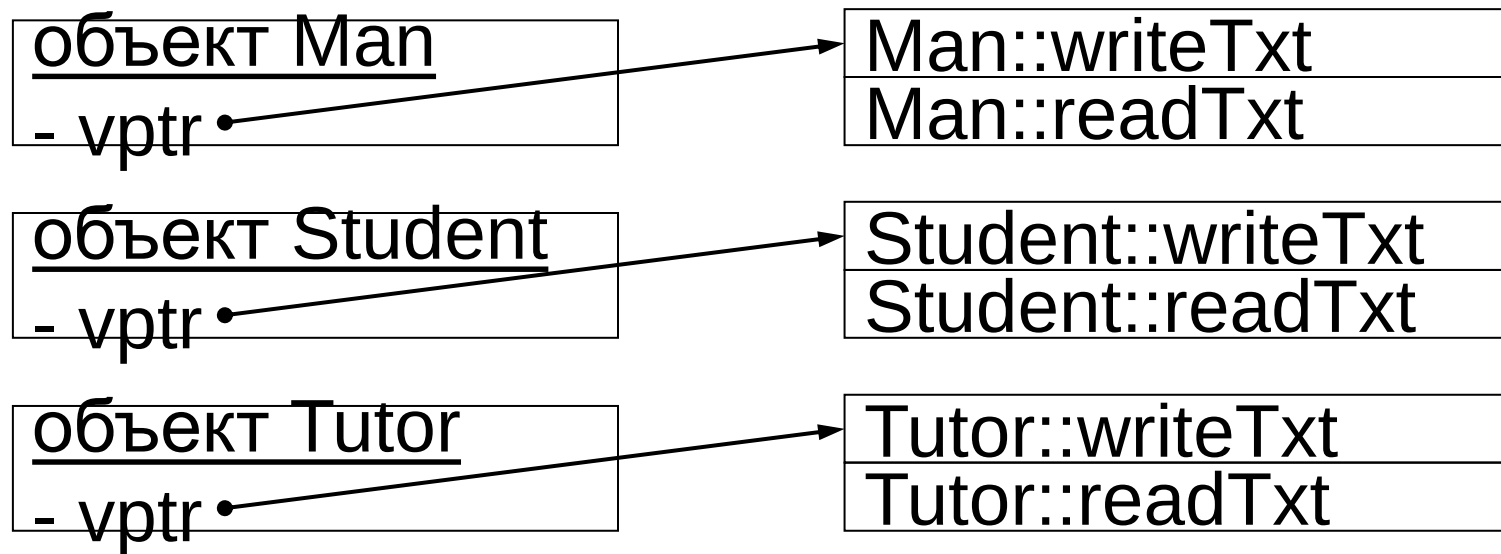
- таблица виртуальных функций VTABLE для каждого класса, содержащего виртуальные функции
- скрытое хранение указателя VPTR на VTABLE в каждом экземпляре класса
- автоматическая генерация кода доступа к функции через VTABLE в точке вызова полиморфной функции

Размещение в памяти

- классы без данных приводятся компилятором к ненулевому размеру за счет вставки фиктивной переменной
- каждый экземпляр класса с виртуальными функциями хранит скрытый указатель VPTR (перед данными)
- в классах без данных с виртуальными функциями место фиктивной переменной занимает указатель VPTR

Схема хранения VTABLE

таблица виртуальных функций



Инициализация указателя VPTR

- виртуальные функции не должны вызываться до инициализации VPTR
- инициализация VPTR происходит в конструкторе (в т.ч. умолчательном)

Наследование и VTABLE

- при создании экземпляра класса в таблицу заносится адреса определенных в классе виртуальных функций
- при добавлении в производном классе новых виртуальных функций таблица расширяется

Повышающее приведение и объекты

- повышающее приведение типа применимо только к адресам (ссылкам)
- если в точке вызова тип точно известен, используется раннее связывание (как типичное поведение компилятора)

Эффективность использования виртуальных функций

- минусы
 - дополнительные инструкции для каждого вызова виртуальной функции
 - дополнительный код для инициализации
 - дополнительная память для хранения
- плюсы
 - повышение гибкости программ
 - повышение эффективности работы программистов

ваши вопросы

Чисто виртуальная функция

- объявление начинается с `virtual` и заканчивается `"= 0"`
- должна реализовываться в производном классе

пример:

```
virtual int func() = 0;
```

Объявление чисто виртуальной функции

- резервирует ячейку в таблице виртуальных функций VTABLE
- не заносит конкретного указателя на реализацию функции
- указывает способ использования класса

Абстрактный класс

- содержит нереализованные чисто виртуальные функции
- создание экземпляров абстрактных классов запрещено
- при наследовании от абстрактного класса все чисто виртуальные функции д.б. реализованы

Абстрактный класс как тип

- определяет тип (интерфейс)
- используется в качестве базового в иерархии наследования для классов реализующих данный интерфейс
- используется в реализации алгоритмов над объектами типа

Использование абстрактных классов

- запрещает передачу экземпляров по значению
- гарантирует осуществление повышающего приведения типа через указатель или ссылку

Определение чисто виртуальной функции

- аналогично обычной функции
- класс остается абстрактным
- позволяет вызывать общий код в производных типах

Определение чисто виртуальной функции (пример)

```
class Printable {  
    public:  
        virtual void writeTxt(ostream& os) = 0;  
};  
  
void Printable::writeTxt(ostream& os) {  
    // общий код  
}  
  
// class Man    : public Printable
```


Изменение типа возвращаемого значения

- ВОЗМОЖНО
- функция базового класса
 - возвращает указатель или ссылку на базовый класс
- переопределенная функция
 - может возвращать указатель или ссылку на производный класс

Особенности конструкторов и деструкторов

- код инициализации VPTR вставляется в конструктор
- механизм виртуального вызова не работает в конструкторах и деструкторах

Виртуальный деструктор

- нужен для правильно разрушения объекта производного класса через указатель/ссылку на экземпляр базового класса
- деструктор базового класса обязательно д.б. виртуальным

пример:

```
virtual ~Base ( ) ;
```

Чисто виртуальный деструктор

- имеет право быть
- должен иметь тело
- предотвращает возможность создания экземпляров класса

Однокоренные иерархии

- работа с контейнерами
- отладочный функционал
 - протоколирование
 - сохранение/восстановление данных

Динамический полиморфизм

- реализуется через механизмы виртуальных функции и наследования
- имя функции сопоставляется с кодом функции на этапе выполнения

Полиморфизм

- помогает создавать расширяемые программы
- увеличивает возможности повторного использования кода и упрощает сопровождение
- механизм реализации и использования абстракций

Интерфейсный класс

- содержит только объявления функций
- не содержит данных
- не содержит определений объявленных функций
- все функции д.б. чисто виртуальными

Наследование интерфейса в C++

- реализуется через наследование от интерфейсного (абстрактного) класса
- наиболее разумный случай использования множественного наследования

Наследование в std

- изучите подсистему потоков
- постройте диаграмму наследования

ваши вопросы